Studio Nx

JS 101 — initTocActiveState()와 TODO 주석 정리

2026-10-13

JS 101 — `initTocActiveState()` (개인정보 처리방침 목차에서 지금 읽는 절 표시)와 오래된 TODO 주석 정리

항목	내용
파일	`studionx/frontend/static/js/pages/privacy.js`, `static/css/pages/privacy.css`, `static/js/core/api.js`, `auth.js`, `auth-ui.js`, `pages/home.js`(주석)
작업	P1-23 화면 마무리의 마지막 함수. 목차의 현재 절 표시(선택 기능)와, 구현이 끝났는데 "TODO"로 남아 있던 주석 정리
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	100 홈의 세 함수 · (P1-23 끝, 개요와 시험 문서는 62, 63번)

1. 하는 일

목차의 현재 절 표시

개인정보 처리방침(`privacy.html`)은 12개 절로 길고, 왼쪽(좁은 화면에서는 위)에 목차가 있습니다. 읽는 절에 해당하는 목차 항목에 `aria-current="true"`를 달고, CSS가 글자색과 왼쪽 선(보라색)으로 표시합니다. 마우스를 올렸을 때의 모양과 같습니다.

스크롤 위치	표시되는 목차
맨 위(제목과 첫 문단)	없음
어떤 절의 제목이 화면 위쪽에 옴	그 절 하나만
두 절의 경계가 위쪽 띠에 걸림	아래쪽(막 들어오는) 절
맨 아래	마지막 절(`contact`)

오래된 TODO 주석 정리

`core/api.js`, `auth.js`, `auth-ui.js`에는 함수를 만들기 전 템플릿 때의 `TODO:` 안내가 남아 있었습니다. 구현은 끝났는데도 "TODO"로 보여서 구현이 안 된 것처럼 읽혔습니다. 8곳(`api.js` 1곳, `auth.js` 6곳, `auth-ui.js` 1곳)을 `구현 메모:`로 바꿨습니다. 내용은 그대로이고 코드는 건드리지 않았습니다. `home.js`의 `initLoginForm({ onSuccess: null /* TODO … */ })`는 기본 동작(`next`가 있으면 그 페이지로, 없으면 모달을 닫고 헤더 전환)을 쓰는 것이 맞아서, 주석을 그 설명으로 바꿨습니다. 맨 위의 "템플릿 — 구현은 직접" 안내도 현재 상태에 맞게 고쳤습니다.

2. 언제 쓰나

호출하는 곳	시점
`privacy.js` 맨 아래 `initTocActiveState();`	개인정보 처리방침이 열릴 때 한 번

3. 코드

```
export function initTocActiveState() {
  const links = document.querySelectorAll('.policy__toc a');
  if (links.length === 0 || typeof IntersectionObserver === 'undefined') return;

  const targets = [];   // { link, section, inside }
  for (const link of links) {
    const href = link.getAttribute('href') || '';
    const hash = href.indexOf('#') >= 0 ? href.slice(href.indexOf('#') + 1) : '';
    const section = hash ? document.getElementById(hash) : null;
    if (section) targets.push({ link: link, section: section, inside: false });
  }
  if (targets.length === 0) return;

  const refresh = function () {
    let current = null;
    for (const target of targets) {
      if (target.inside) current = target;   // 띠에 두 절이 함께 걸치면 아래쪽 절
    }
    const atBottom = window.innerHeight + window.scrollY >= document.documentElement.scrollHeight - 2;
    if (atBottom) current = targets[targets.length - 1];
    for (const target of targets) {
      if (target === current) target.link.setAttribute('aria-current', 'true');
      else target.link.removeAttribute('aria-current');
    }
  };

  const observer = new IntersectionObserver(function (entries) {
    for (const entry of entries) {
      for (const target of targets) {
        if (target.section === entry.target) target.inside = entry.isIntersecting;
      }
    }
    refresh();
  }, { rootMargin: '-10% 0px -80% 0px', threshold: 0 });

  for (const target of targets) observer.observe(target.section);
  window.addEventListener('scroll', function () { window.requestAnimationFrame(refresh); }, { passive: t
rue });
}
```

CSS (`privacy.css`, 마우스를 올렸을 때의 규칙에 한 줄 더함)

```
.policy__toc a:hover, .policy__toc a:focus-visible,
.policy__toc a[aria-current="true"] { color: var(--c-ink); border-left-color: var(--c-violet-ink); }
```

최신 문법으로 쓰면 (공부용)

```
export const initTocActiveState = () => {
  const targets = [...document.querySelectorAll('.policy__toc a')]
    .map((link) => ({ link, section: document.getElementById(link.hash.slice(1)), inside: false }))
    .filter((t) => t.section);
  if (!targets.length || !('IntersectionObserver' in window)) return;
```

```
  const refresh = () => {
    const atBottom = innerHeight + scrollY >= document.documentElement.scrollHeight - 2;
    const current = atBottom ? targets.at(-1) : targets.findLast((t) => t.inside);
    targets.forEach((t) => t.link.toggleAttribute('aria-current', t === current));
  };
  const io = new IntersectionObserver((entries) => {
    entries.forEach(({ target: el, isIntersecting }) => { targets.find((t) => t.section === el).inside =
isIntersecting; });
    refresh();
  }, { rootMargin: '-10% 0px -80% 0px' });
  targets.forEach((t) => io.observe(t.section));
  addEventListener('scroll', () => requestAnimationFrame(refresh), { passive: true });
};
```

최신 문법	실제 코드	뜻
`link.hash`	`href.indexOf('#')`와 `slice`	링크 주소에서 `#` 뒤 부분을 바로 꺼냅니다
`targets.at(-1)`	`targets[targets.length - 1]`	마지막 요소
`targets.findLast(…)`	반복에서 마지막으로 덮어쓰기	조건에 맞는 마지막 요소
`toggleAttribute('aria-current', 조건)`	`setAttribute`와 `removeAttribute`	속성을 켜고 끕니다
`.map().filter()`, `{ link, section }`	`for`와 `if`, `{ link: link, … }`	목록 변환과 줄여 쓴 객체

문법 풀이

식	뜻
`rootMargin: '-10% 0px -80% 0px'`	관찰 범위를 위에서 10%, 아래에서 80% 잘라 냅니다. **위에서 10% ~ 20% 사이의 얇은 띠**만 남아서, 그 띠에 걸친 절이 "지금 읽는 절"입니다
`entry.isIntersecting`	그 절이 띠에 걸쳐 있는지입니다
`window.innerHeight + window.scrollY >= document.documentElement.scrollHeight - 2`	화면 아래끝이 문서 끝에 닿았는지(맨 아래인지)입니다. 2는 소수점 오차를 위한 여유입니다
`window.requestAnimationFrame(refresh)`	다음 화면 갱신 직전에 한 번 실행합니다. 스크롤이 초당 수십 번 일어나도 화면 갱신당 한 번만 계산합니다
`{ passive: true }`	이 처리기가 스크롤을 막지 않는다고 브라우저에 알려 스크롤이 매끄럽게 합니다
`href.slice(href.indexOf('#') + 1)`	목차 링크가 `privacy.html#who`처럼 파일 이름을 포함해서, `#` 뒤의 `who`만 뽑습니다 (세로 앵커의 `#service`와 다른 점입니다)

4. 왜 이렇게

선택	이유
화면 **위쪽 띠**(10% ~ 20%) 기준	처리방침은 글을 위에서 아래로 읽는 문서라서, 절 제목이 화면 위쪽에 닿았을 때가 "이 절을 읽기 시작함"입니다. 세로 앵커(홈)는 섹션이 화면 한 장이라 가운데 기준이었습니다
경계에서는 **아래쪽** 절	띠가 얇아도 두 절이 함께 걸칠 수 있습니다. 새로 들어오는 절을 표시하는 것이 자연스럽습니다 (시험에서 위쪽 절을 고르는 첫 구현이 틀렸음을 발견해 고쳤습니다)
맨 아래에서는 마지막 절	마지막 절들은 문서 끝에 막혀서 위쪽 띠까지 올라오지 못합니다. 그대로 두면 `children`, `breach`, `changes`, `contact`는 영영 표시되지 않습니다

선택	이유
스크롤 처리기도 함께 둠	맨 아래 판단은 "절이 띠에 걸쳤는가"와 무관하게 스크롤 위치로만 알 수 있어서, 관찰기가 알려 주지 않는 순간에도 다시 계산해야 합니다
글머리에서는 표시 없음	제목과 첫 문단은 어느 절도 아닙니다
선택 기능이라 실패해도 조용히 끝냄	처리방침은 표시가 없어도 읽는 데 지장이 없어야 합니다. `IntersectionObserver`가 없으면 그냥 끝냅니다
오래된 `TODO`를 `구현 메모`로	문서 내용은 그대로이고, 구현이 안 된 곳처럼 읽히는 것만 고쳤습니다. 코드는 건드리지 않았습니다

5. 시험 (실제 서버, 헤드리스 Chrome, 마우스로 조작, 12개)

#	한 일	결과
P00	목차 링크	12개 ✓
P01	맨 위	표시 없음 ✓
P02	각 절의 제목을 화면 위쪽 띠에 놓음 (스크롤이 닿는 8개 절)	그 절의 목차 **하나만** 표시 ✓
P03	맨 아래로	마지막 절 `contact` 표시 (띠에 닿지 못해도) ✓
P04	다시 맨 위로	표시가 사라짐 ✓
P05	목차의 `rights`를 마우스로 누름	그 절로 이동하고 표시됨 ✓
P06	표시된 항목과 보통 항목	글자색과 왼쪽 선 색이 다름 (CSS) ✓
P07	표시 개수	한 번에 하나 ✓
P08	폭 390 (`rights` 절)	동작 ✓
P09	함수를 두 번 더 부름	표시는 하나 이하, 오류 없음 ✓
P10	가리키는 절이 없는 목차만 있음	오류 없이 끝남 ✓
P11	`IntersectionObserver`가 없음	오류 없이 끝남 ✓

12개 모두 통과했습니다. 처음 돌렸을 때 2개가 실패했습니다. 하나는 시험이 문서 끝에 가까운 절(`children` 이후)을 위쪽 띠까지 스크롤하려 했는데 문서가 거기까지 못 올라가는 것이었고(시험이 닿는 절만 확인하도록 고침), 다른 하나는 절 경계를 띠에 놓았을 때 **위쪽 절이 표시된 것**이었습니다. 이쪽은 시험이 아니라 함수의 선택이 맞지 않아서, 경계에서는 아래쪽 절을 고르도록 함수를 고쳤습니다.

`TODO` 정리 확인: `static/js` 아래에서 남은 "TODO"는 `core/modal.js`(오래된 브라우저용 대체 코드, 필요 없음), `core/sentry-init.js`(프론트 Sentry, Phase 1 범위 밖), 시퀀서 파일들(Phase 2)뿐입니다.